

UNIX Shell Programming

- #1. Display report showing - the full name of all users currently connected
- the nr. of processes belonging to each of them.

prob1.sh

#!/bin/bash

currentlyConnected=\$(cat who.fake | cut -d' ' -f1 | sort | uniq)

for userLi in \$currentlyConnected; do

 nameCompleet=\$(grep "\$userLi" passwd.fake | cut -d':' -f 5)

 numarProcese=\$(grep "^\$userLi" ps.fake | wc -l)

 echo "\$nameCompleet \$numarProcese"

done

- #2. Find recursively in a directory all ".c" files having > 500 lines.
? Stop after finding 2 such files.

prob2.sh

#!/bin/bash

if [\$# -eq 1]; then

 echo "Insufficient args"

 exit 1

fi

count = 2

while IFS = read -r f; do

(=)

 if [\$(wc -l < \$f) -gt 500]; then

 echo "\$f"

 count = \$(count - 1)

 if [\$count -eq 0]; then

 break

 fi

fi

done < <(find "\$1" -type f -name "*.c")

#6. Display-session count } of all users who logged into the system this
- full names } month, sorting output by session count in
descending order.

```
#!/bin/bash
```

```
sessionSnapshot = $(cat last.fake)
```

```
theUsers = $(while read i; do
```

```
zuaJoined = $(echo $i | awk '{print $4}')
```

```
if ["$zuaJoined" = "Sun"]; then
```

```
echo "$i" | grep -o -E "^[^ ]+"
```

```
fi
```

```
done <<< "$sessionSnapshot" | sort | uniq)
```

```
for userI in $theUsers; do
```

```
numberSessions = $(echo "$sessionSnapshot" | grep "^$userI:" | wc -l)
```

```
theFullName = $(grep "^$userI" passwd.fake | cut -d':' -f 5)
```

```
echo "$numberSessions $userI $theFullName"
```

```
done | sort -m -n -k 1
```

numeric reverse key 1 (nrSessions)

#7. Consider file containing a username on each line.

Generate a comma-separated string with email addresses.

Append @scs.ubc... at the end of each username.

Don't end in comma!

```
#!/bin/bash
```

```
emails = ""
```

```
while read i; do
```

```
grep -q "^$i:" passwd.fake → checks if user exists
```

```
if [ $? -eq 0 ]; then → runs only if grep found a match.
```

```
emailAddress = "$i@scs.ubcduj.ro"
```

```
emails = "$emails$emailAddress,"
```

```
fi
```

```
done < thefisier → read input from file.
```

```
echo $emails | head -c -2
```

```
echo
```


8. Display all mounted file systems who are either smaller than 1GB or have less than 20% free space.

Solve 8. awk

```
? treat as nr, not text
{
    size_mb = $2 + 0
    use_percent = $5 + 0
    if (size_mb < 1024 || use_percent > 80) {
        print $6
    }
}
```

> \$ cat df.fake | awk -f solve8.awk

- #9. Find in given directory hierarchy all duplicate files and display their paths checksums to detect if two files are identical?

prob9.sh

#!/bin/bash

if [\$# -eq 0]; then

echo "No args given."

directory = "."

elif [\$# -eq 1]; then

if [-d "\$1"]; then

directory = "\$1"

else

echo "We need an existing directory."

exit 1

fi

else

echo "Too many args!"

exit 1

fi

allFiles = \$(find "\$directory" -type f)

fileData = \$(while read -r file; do

cksum "\$file"

done <<< "\$allFiles")


```
duplicates = $(echo "$fileData" | awk '{print $1, $2}' | sort | uniq -d)
```

echo "Files that have duplicates:"

```
while read -r sum size; do
    echo " --- "
    echo "$fileData" | awk -v s = "$sum" -v sz = "$size" '
        $1 == s && $2 == sz {print $3}'
done <<< "$duplicateKeys"
```

create awk variable(s) with shell value of sum

create awk variable(s) with shell value of size

checksum matches

size matches

filename path

#3. Find recursively in a directory all the files with the extension ".log" and sort their lines (replace original file with sorted content).

```
#!/bin/bash
if [ $# -eq 0 ]; then
    echo "No args given"
    exit 1
elif [ $# -eq 1 ]; then
    if [-d "$1"]; then
        directory = "$1"
    else
        echo "We need an existing directory"
        exit 1
    fi
else
    directory = './'
fi
files = $(find "$directory" -name "*.log")
# echo "${files[@]}"
for file in "${files[@]"; do
    sort $file -o $file
done
```

no spaces!!!

4. Find recursively in given directory all symbolic links, report those that point to files/directories that no longer exist.

-L → test if a path is a symbolic link

-e → test if it exists

```
#!/bin/bash
```

```
if [ $# -eq 0 ]; then
```

```
    echo "We need arguments."
```

```
elif [ $# -eq 1 ]; then
```

```
    if [ test -d "$1" ]; then
```

```
        directory = "$1"
```

```
    else
```

```
        echo "We need existing directory"
```

```
        exit 1
```

```
    fi
```

```
else
```

```
    directory = './'
```

```
fi
```

```
paths = ( $(find $directory)
```

```
for path in "${paths[@]}; do
```

```
    # echo "Checking: $path"
```

```
    if [ -L "$path" ]; then
```

```
        # echo "symlink"
```

```
        if ! [ -e "$path" ]; then
```

```
            echo "broken → $path"
```

```
        fi
```

```
    fi
```

```
done
```


#5. Receive dangerous program names as command-line arguments.

Monitor all the processes in the system.

Whenever a program dangerous is run, script will kill it. $\Rightarrow$ message

```
#!/bin/bash
```

```
if [ $# -eq 1 ]; then
```

```
    danger=$1
```

```
else
```

```
    echo 'Give name of dangerous program'
```

```
    exit 1
```

```
fi
```

```
while ((1>0)); do
```

$\downarrow$?

```
    processes=$(cat ps.fake | grep "$danger" | awk '{print $2}' | head -n -2)
```

```
    for pr in "${processes[@]}"; do
```

```
        if [ $$ != $pr ]; then
```

```
            echo "$danger" was killed
```

```
            kill "$pr"
```

```
        fi
```

```
    done
```

```
    sleep 1
```

```
done
```

(()) $\rightarrow$ math

[] $\rightarrow$ conditions

() $\rightarrow$ subshell

#6. Find recursively in a dir. all the files that have write permissions for everyone. Display names and permissions before and after removing w for everybody.

chmod's symbolic permissions mode, (not octal mode)

human-readable form

```
#!/bin/bash
```

```
if [ $# -ne 1 ]; then
```

```
    echo "Give 1 directory"
```

```
    exit 1
```

```
fi
```

```
if [ ! -d "$1" ]; then
```

```
    echo "Must be an existing directory"
```

```
    exit 1
```

```
fi
```

```
for file in $(find "$1" -type f
```

```
-perm -222); do
```

```
    before=$(stat -c "%A" "$file")
```

```
    echo "$file : before = $before"
```

```
    chmod a-w "$file"
```

```
    after=$(stat -c "%A" "$file")
```

```
    echo "$file : after = $after"
```

```
done
```

$\rightarrow$ all users